Spilling the T: The Hidden Cost of Single-Qubit Rotations

YQuantum 2026, QuEra Challenge

Shuhul Mujoo,[1] Nandana Madhukara,[1] Thuwaragesh Jayachandran,[1] Joshua Mu,[1] and Emiliano Nolasco[2]
(Team Calbits)
[1] *California Institute of Technology*
[2] *San Diego State University*

I. INTRODUCTION

On paper, applying an operation to a single qubit looks much easier than entangling two qubits together. It is just one qubit, after all! However, in the real world of quantum computing, this intuition quickly falls apart. In this report, we will build a technical case around a surprising claim: a 1-qubit gate can actually be much harder to implement than a 2-qubit gate.

We will investigate this by looking at a specific family of 1-qubit rotations, $R_z(\pi/2^n)$. While standard 2-qubit gates like the CNOT are relatively straightforward to apply in error-corrected systems, making small and precise 1-qubit rotations requires massive sequences of discrete gates to achieve high precision.

In the following sections, we will walk through the entire journey of building these tiny rotations. First, we will look at how to synthesize them physically using the standard Clifford+T gate set. Next, we will introduce the painful reality of fault tolerance, where our most crucial tool (the T gate) cannot be applied directly and must be teleported into the circuit using extra auxiliary qubits. Finally, we will move from physical qubits to logical qubits using the $[[7, 1, 3]]$ Steane code to see how these overhead costs multiply when we need to protect our data from noise.

II. UNIVERSAL GATES

Universal gates are a discrete set of gates that can implement any gates in a method of computation. In classical computation the NAND gate can alone do the job, but this is a bit tricky in the quantum case. The reason is simple. Suppose you start from $|0\rangle$. You can rotate your qubit to literally any point on the Bloch sphere, and each point would correspond to a unique transformation and so a unique gate! The continuous nature of the Bloch sphere evades the existence of a discrete finite set of universal gates in quantum computation [1, 2] (Figure 1).

The best we can do is to have a finite discrete set of quantum gates with which we can approximate any unitary operator to any required accuracy. A better accuracy needs a larger number of these gates to be implemented. In this project, we will be using the following standard universal set of quantum gates: Clifford + $T := \{H, S, T, CNOT\}$. Let's meet each of them. (Verification of these gate actions is documented in `simulate_gates.py`)

The $CNOT$ is the only 2-qubit gate in the set. It is well known that a set of only one-qubit gates cannot be universal [1, 4] and $CNOT$ helps counteract this. The 1-qubit members are slightly more interesting because they all move around the Bloch sphere in deceptively simple manner. T rotates the qubit about the z-axis by $\frac{\pi}{4}$ and $S = T^2$ making it a $\frac{\pi}{2}$ rotation about the z-axis. Finally, the Hadamard gate H is a π rotation about the axis halfway between x and z-axes. The avengers are assembled in Figure 2.

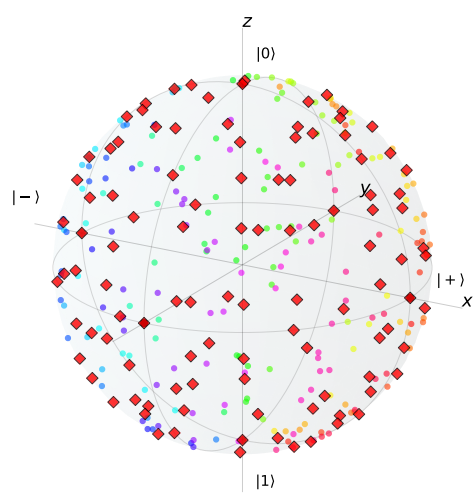


FIG. 1. The Bloch sphere is continuous. Every point corresponds to a unique qubit state. Colored dots represent the infinite number of possible states. Red diamonds mark the approximately 100 distinct states reachable by Clifford+T sequences of depth ≤ 4. Even with T generating irrational phases, short sequences only reach a sparse discrete subset. Longer sequences fill in more points to create a dense set. However, they never cover the sphere exactly. They only approximate it to high precision (Solovay-Kitaev theorem [2, 3]).

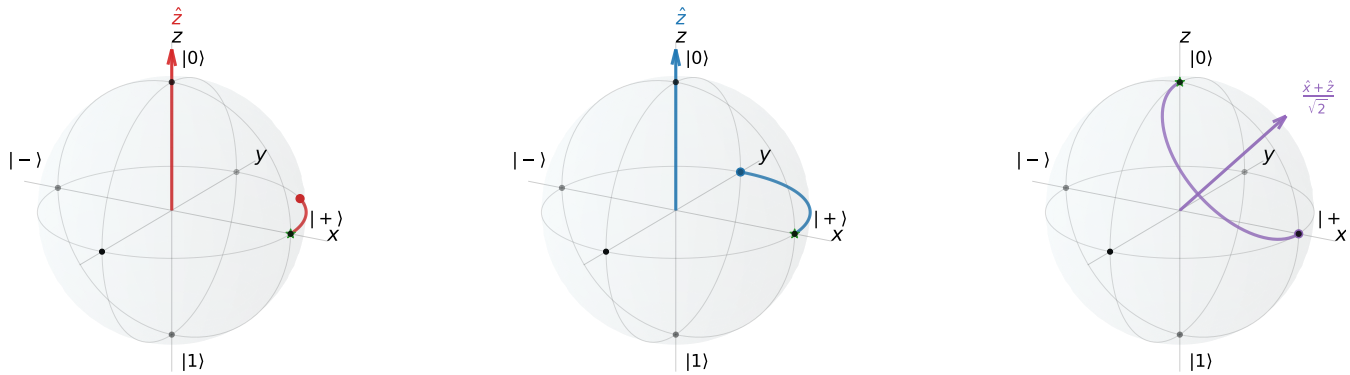


FIG. 2. The three 1-qubit gates in Clifford+T visualized on the Bloch sphere. The left image shows $T = R_z(\pi/4)$ which is a 45° rotation about $\hat{z}$. The green star is the initial state $|+\rangle$ and the red dot is the final state $T|+\rangle$. The center image shows $S = T^2 = R_z(\pi/2)$ which is a 90° rotation about $\hat{z}$ mapping $|+\rangle$ to $|i\rangle$. The right image shows H which is a 180° rotation about the $(\hat{x}+\hat{z})/\sqrt{2}$ axis swapping $|0\rangle$ and $|+\rangle$.

III. T-URNING THE QUBITS WITH T

Having empowered ourselves with the universal set Clifford + T, let's now try to move our qubit through the Bloch sphere. Let's just focus on one qubit rotations about the z-axis, $R_z(\alpha)$. How hard can it be? After all, the computational basis states $|0\rangle$ and $|1\rangle$ are aligned along the z-axis and so individually they are both unaffected (upto a global phase). Well, quantum is always tricky!

Rotating a general qubit residing at (θ, ϕ) by α about the z-axis takes the qubit to $(\theta, \phi + \alpha)$. Therefore, we can write,

$$R_z(\alpha) = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\alpha} \end{pmatrix} \tag{1}$$

Specifically, we consider the family of

$$\alpha = \frac{\pi}{2^n} \quad \text{for} \quad n = 0, 1, 2, 3, 4, 5 \tag{2}$$

in this problem. Note that $R_z(\alpha_1+\alpha_2) = R_z(\alpha_1)R_z(\alpha_2)$. This means that with the availability of $T = R_z(\frac{\pi}{4}) = R_z(\frac{\pi}{2^2})$ for $n = 2$, we can easily implement $n = 1$ and $n = 0$ by repeated applications of T. (implemented explicitly in `exact_decompositions.py`) Concretely,

$$R_z\left(\frac{\pi}{2^2}\right) = T \tag{3}$$

$$R_z\left(\frac{\pi}{2^1}\right) = T^2 = S \tag{4}$$

$$R_z\left(\frac{\pi}{2^0}\right) = T^4 = S^2 = Z. \tag{5}$$

Looks very promising. How about $n > 2$? As it turns out, we can only approximate these gates with our universal set.

Of course, our set limits us to $\frac{\pi}{4}$ rotations about the z-axis, and so we can not achieve $R_z(\frac{\pi}{8})$ with S and T alone. H is a reflection about the halfway axis between Z and X and so intuitively it seems quite unlikely that this will help, unless supported by some other gate that can do a $\frac{\pi}{8}$ rotation. This indeed turns out to be true as proved in [5].

We will first present our synthesis procedures for $n > 2$ and then follow it up with optimal approximations to these unitaries from the literature.

A. Approximation of $R_z(\frac{\pi}{2^n})$ for $n > 2$

We propose and implement a naiive scheme for $n = 3$, which we build upon to obtain approximations for $n = 4$ and $n = 5$. Since we are limited to the one qubit gates H, S and T, and $S = T^2$, we begin with a brute-force approach to search for the best approximation of an l-gate implementation $\{H, T\}^l$ to $R_z(\frac{\pi}{8})$. For instance, $l = 3$ would contain the following gates: $\{HHH, HHT, HTH, HTT, THH, THT, TTH, TTT\}$. We iterate over all 2^k gates for all $1 \le k \le l$ and find the gate corresponding to the least distance. Considering runtime and the efficiency of the gate implementations, we choose $l = 23$ and obtain the 19-gate,

$$\begin{aligned} V &= HTHTHTTTHTHTTTHTHTH \\ &= HTHTHSTHTHSTHTHTH, \end{aligned}$$

as the best approximation to $R_z(\frac{\pi}{8})$ with a trace distance of $d(R_z(\frac{\pi}{8}), V) = 0.03972141$. We note that this required the implementation of a minimum of 7 T gates.

An explicit demonstration of V is given in Figure 3. The gate is applied to $|+\rangle$ and results in a state $V|+\rangle$ with $(\theta, \phi) = (88.77^\circ, 22.03^\circ)$. Now, we make the following important observation that we demonstrate with the $|+\rangle$ state to gather some intuition. $V = R_{\hat{n}}(\beta)$ for some unit vector $\hat{n}$ and β. For $|+\rangle$ we know that $\beta \approx 22.03^\circ$

and $\hat{n}$ is very close to $\hat{z}$. Successive applications of V are going to rotate the state about $\hat{n}$, while keeping the state on the boundary of the plane perpendicular to $\hat{n}$, which is a circular trajectory. Since β isn't an exact divisor of 2π (it's probably an irrational fraction of 2π), the state never returns to where it started. Therefore, with enough repetitions, we will be able to find a gate that takes $|+\rangle$ closer to anywhere in that circle.

Now, note that $n = 4$ and $n = 5$ correspond to smaller angles. Since $n = 3$ for $\frac{\pi}{8}$ is a good approximation as we saw, $n = 4$ and $n = 5$ approximation points on the circle must be between $|+\rangle$ and $V|+\rangle$ and so still must have $\theta \approx 90°$. Therefore, these should provide a good approximation for $R_z(\frac{\pi}{2^n})$ for $n > 3$. Essentially, we are claiming that,

$$\exists\, k \quad \text{such that} \quad V^k = (R_{\hat{n}}(\beta))^k \approx R_z\left(\frac{\pi}{2^n}\right) \quad \text{for} \quad n > 3 \tag{6}$$

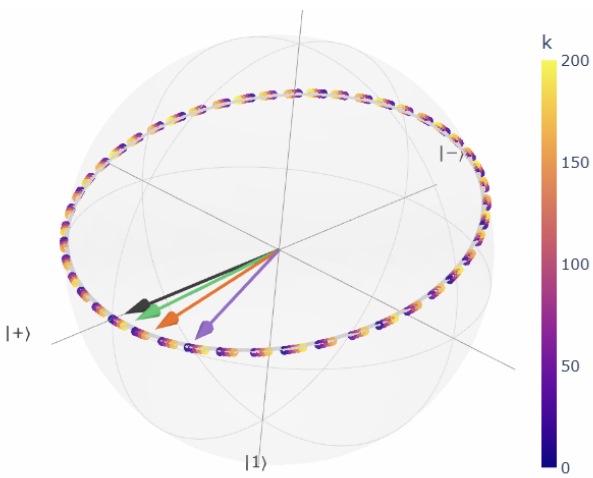


FIG. 3. Demonstration of V^k as an approximate gate for $R_z(\frac{\pi}{2^n})$ for $n = 4, 5$. The black arrow shows the initial state $|+\rangle$ and the purple arrow shows $V|+\rangle$, where V is the approximator for $R_z(\frac{\pi}{8})$. Repeated application of V takes the state around the circular trajectory shown. With the rotation angle not being a rational multiple of 2π, $V^k|+\rangle$ will span all points on the circle, if we let $k \to \infty$. Thus, we can find k for which we find a good approximator for $n = 4$ and 5 since these are smaller angles than the one V approximated. After running 200 times, the orange arrow ($k = 157$) and the green arrow ($k = 188$) represent the best approximations for $n = 4$ and $n = 5$ respectively.

Even though this scheme works, it is clear why this will be inefficient and expensive to implement. Each implementation of V requires 7 T gates and 10 other gates. For V^k, this means $17k$ gates in total with $10k$ T gates. Figure 4 shows the distance of the approximation as a function of k. We observe repeated local minima, but the minimal distance isn't decreasing that much as seen. Therefore, accounting for cost efficiency, it might be optimal to choose the first minimum itself. This turns out to be $k = 16$ for both $n = 4$ and $n = 5$.

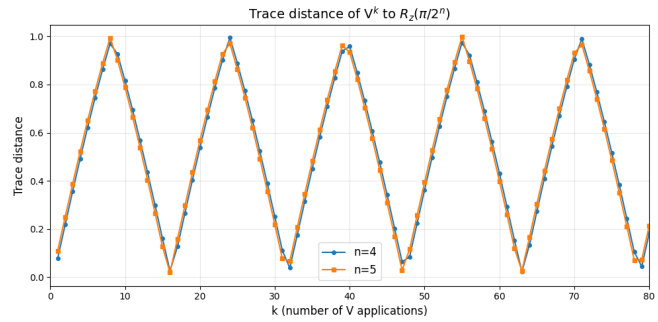


FIG. 4. Trace distance $d(R_z(\frac{\pi}{2^n}), V^k)$ as a function of k for $n = 4$ and 5. We obviously see a repeated pattern and also that both $n = 4$ and $n = 5$ very closely follow each other since their angles differ by only 5.625°. Even though exploring higher k potentially results in better approximations, as is shown in Figure 3, the improvement is not justified by the complexity.

B. Approximation using optimal algorithms in literature

We use the following two algorithms: the Solovay-Kitaev (SK) algorithm (implemented from scratch) and the Ross-Selinger (RS) algorithm (via `pygridsynth`) [both evaluated in `approximate_synthesis.py`].

SK is a recursive algorithm that converts a coarse approximation into a fine one. At each recursion level, it decomposes the residual error into a balanced group commutator $[W, V] = WVW^\dagger V^\dagger$ of two "easier" rotations, which are themselves recursively approximated. If V and W are close to the identity and the base error is ε_0, then after d levels the error is $\varepsilon_d \sim \varepsilon_0^{(3/2)^d}$. However, the gate count grows as:

$$\text{Gate count} = O\left(\log^{3.97}\left(\frac{1}{\varepsilon}\right)\right).$$

SK implementation details

- Initial guess: BFS over $\{H, T, T^\dagger\}$ to depth 15, yielding 5,822 distinct unitaries (built in < 1s).
- New guess: $UU_{n-1} \approx \tilde{W}_{n-1}\tilde{V}_{n-1}\tilde{W}_{n-1}^\dagger\tilde{V}_{n-1}^\dagger$
- Tested depths: 0-4.

On the other hand, RS uses number theory to directly solve a Diophantine approximation problem over the ring $\mathbb{Z}[\omega]$ (where $\omega = e^{i\pi/4}$), finding near-optimal decompositions:

$$T\text{-count} = 3\log_2\left(\frac{1}{\varepsilon}\right) + O(\log\log(1/\varepsilon)).$$

(circuit definitions and Quirk visualizations provided in `rs_circuits.py`) This is provably optimal up to an additive constant.

Here are the results (see Figure 5):

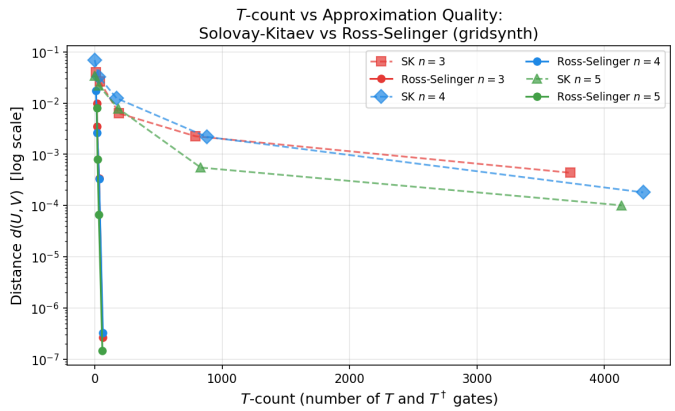


FIG. 5. T-count vs approximation distance. Ross–Selinger (solid) achieves orders-of-magnitude better efficiency than Solovay–Kitaev (dashed) at every precision level.

n	Method	Gates	T-count	Distance	
3	SK (depth 4)	6,783	3,735	0.0004	
3	RS (ε=10^{-3})	88	34	0.0003	110× fewer T
4	SK (depth 4)	7,842	4,304	0.0002	
4	RS (ε=10^{-3})	90	34	0.0003	127× fewer T
5	SK (depth 4)	7,624	4,136	0.0001	
5	RS (ε=10^{-3})	81	32	7×10^{-5}	129× fewer T

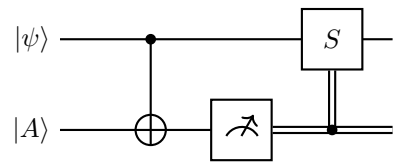
The Solovay-Kitaev algorithm helps us build a specific rotation by repeating a simple set of steps over and over. However, this method is not very efficient for tiny angles. As the rotation $\pi/2^n$ gets smaller, our starting set of gates is a poor match because those short sequences are far away from the target. To fix this, we have to run many more recursive steps, which quickly pushes the total gate count into the thousands. While this eventually reaches the right precision, it creates much larger circuits than newer methods. In contrast, the Ross-Selinger algorithm is a much more efficient way to handle these precise rotations. Its T gate count depends mostly on the precision ε rather than the angle itself. For a precision of 10^{-3}, this approach already requires 34 T gates no matter how small the angle is.

Why T count matters

At comparable accuracy ($d \approx 3 \times 10^{-4}$), Ross–Selinger uses ∼34 T-gates while SK uses ∼4,000. As we will see in the next sections, every T gate eventually costs an ancilla qubit, a CNOT, and a measurement. The synthesis algorithm's T-count directly determines the physical resource cost of the rotation.

IV. TEA IS EXPENSIVE!

In a fault-tolerant architecture, T gates cannot be applied directly to logical qubits because T is not transversal in most error-correcting codes. Only Clifford gates (H, S, CNOT) are allowed on the data qubit. The solution to this problem is to use magic state injection (circuit shown below):



where $|A\rangle = (|0\rangle + e^{i\pi/4}|1\rangle)/\sqrt{2} = TH|0\rangle$ is the magic state.

Here is a short proof of correctness. Initial state:

$$|\Psi_0\rangle = (\alpha|0\rangle + \beta|1\rangle) \otimes \frac{1}{\sqrt{2}}(|0\rangle + e^{i\pi/4}|1\rangle)$$

$$= \frac{1}{\sqrt{2}}(\alpha|00\rangle + \alpha e^{i\pi/4}|01\rangle + \beta|10\rangle + \beta e^{i\pi/4}|11\rangle)$$

Then applying the CNOT,

$$\begin{aligned}|\Psi_1\rangle &= \frac{1}{\sqrt{2}}\big(\alpha|00\rangle + \alpha e^{i\pi/4}|01\rangle \\ &\quad + \beta e^{i\pi/4}|10\rangle + \beta|11\rangle\big) \\ &= \frac{1}{\sqrt{2}}\Bigg[\underbrace{(\alpha|0\rangle + \beta e^{i\pi/4}|1\rangle)}_{T|\psi\rangle} \otimes |0\rangle \\ &\quad + \underbrace{(\alpha e^{i\pi/4}|0\rangle + \beta|1\rangle)}_{\substack{T^\dagger|\psi\rangle \\ \text{(up to global phase)}}} \otimes |1\rangle\Bigg]\end{aligned}$$

Finally, the second qubit measurement. If the measurement is 0, then the data qubit must be $T|\psi\rangle$, so that case works. If the measurement is 1, then the data qubit is $T^\dagger|\psi\rangle = \alpha|0\rangle + \beta e^{-i\pi/4}|1\rangle$. To fix this, apply S gate:

$$\begin{aligned}S(T^\dagger|\psi\rangle) &= \alpha|0\rangle + \beta e^{i(\frac{\pi}{2}-\frac{\pi}{4})}|1\rangle \\ &= \alpha|0\rangle + \beta e^{i\pi/4}|1\rangle = T|\psi\rangle\end{aligned}$$

In both cases, the T gate has been applied to the data qubit $|\psi\rangle$. Besides the algebraic proof, we also verified the procedure through code.

Verification

Verified via Bloqade simulator: all test states ($|0\rangle, |1\rangle, |+\rangle, |i\rangle$) produce overlap = 1.000000 between direct T and injected T. Double injection $T^2 = S$ also verified. (the injection protocol simulation is in `magic_state_injection.py`)

Now that we have the rotations on the main data qubits, we benchmark them using the previous distance metric. We approximate each $R_z(\pi/2^n)$ on the data qubit by replacing every T gate in the RS sequence with the injection protocol above, then measure the distance $d(R_z(\pi/2^n), V_{\text{injected}})$. All circuits are simulated within PyQrack's capabilities.

n	Target	Exact?	Ancillas	CNOTs	Distance
0	$R_z(\pi) = Z$	Yes	0	0	0
1	$R_z(\pi/2) = S$	Yes	0	0	0
2	$R_z(\pi/4) = T$	Yes	1	1	0
3	$R_z(\pi/8)$	No	34	34	3.4×10^{-4}
4	$R_z(\pi/16)$	No	34	34	2.7×10^{-4}
5	$R_z(\pi/32)$	No	32	32	7×10^{-5}

The most important point is that the distances are identical to Part II (with T gate). This shows that magic state injection introduces no additional error beyond the Clifford+T synthesis approximation.

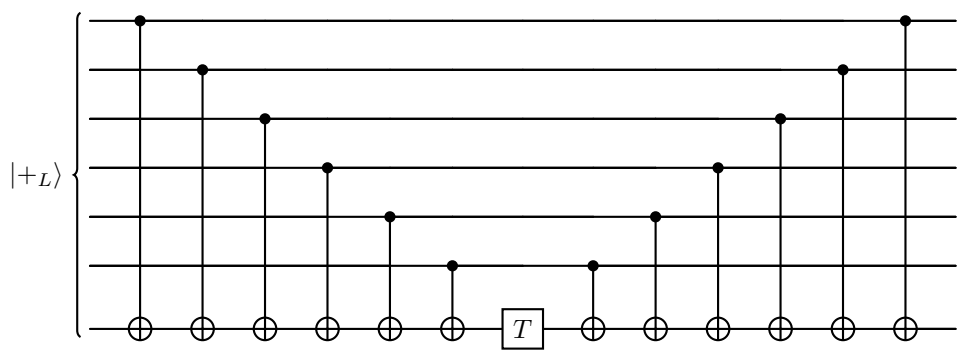
Lets track the new costs that appear. Implementing a sequence of k T gates via magic state injection requires exactly k ancilla qubits, k CNOT gates, and k measurements. Since each individual T gate is replaced by a single instance of these resources, the total overhead for the entire synthesis sequence scales linearly with k.

V. T-IME TO ENCODE!

The first step was to encode the logical qubits $|0_L\rangle$ and $|1_L\rangle$ as certain states for the seven qubits. The circuit for $|0_L\rangle$ was given in the problem and $|1_L\rangle$ was obtained with the expression $|1_L\rangle = X^{\otimes 7}|0_L\rangle$ since $X^{\otimes 7}$ is the logical X operator.

Now in the [[7, 1, 3]] Steane code, all the Clifford gates, H, S, and CNOT, are transversal in the sense that the logical versions of these operators can be implemented by applying single qubit gates on the physical qubits. However, according to the Eastin-Knill it is impossible to implement a universal gate set transversely and in our case, this shows up as T not being able to be implemented transversely. (Implementations and testing of this logical code can be found in `logical_qubit.py`)

To circumvent this, we perform magic state injection where we start with a logical magic state and use it to teleport a logical T onto the qubits containing the data. In reality, this logical magic state $|A_L\rangle$ is found through distillation, but in our case, we will just assume we are given it. For simulation purposes, we will prepare the logical magic state just by applying the (non fault-tolerant) logical T onto $|+_L\rangle$ (similar to the previous part). The code verifying this state is found in `verify_magic_state.py`. This can be visualized in the following circuit:



The intuition for this circuit is the following: Let $|x_1, ..., x_7\rangle$ be one of the computational basis vectors. After the first set of CNOTs, this turns into

$$|x_1, ..., x_6, x_1 \oplus \cdots \oplus x_7\rangle\,,$$

so the last qubit holds the parity which we will call p. Next, the T on the last qubit gives us a phase of $e^{i\pi p/4}$ and the remaining CNOTs brings the state back to the original basis vector. Therefore, the whole transformation is

$$|x_1, ..., x_7\rangle \mapsto e^{i\pi p/4}\,|x_1, ..., x_7\rangle\,.$$

Now in the Steane code, the logical 0 has even parity (four 1's) and the logical 1 has odd parity (three 1's). Therefore, under this circuit, we have

$$|0_L\rangle \mapsto |0_L\rangle \quad |1_L\rangle \mapsto e^{i\pi/4}\,|1_L\rangle\,,$$

which is just a logical T gate.

Finally, we discuss benchmarking the logical implementation of the Z-rotation synthesis in the Steane code. Assuming we have access to logical magic states from distillation, all the logical operations are implemented transversely so the circuit depth doesn't change. This is because transversal gates can be implemented as parallel

single-qubit gates. The main overhead just comes from an increase in the number of physical qubits: the number of ancillas and number of CNOTs will be multiplied by 7.

VI. TEAM STAR

A. Part 1

With the STAR (Space-Time efficient Analog Rotation) architecture, we have a framework from which to obtain stabilizer readings of the logical qubits encoded in the code (We implemented the base distance-3 surface code and validation routines in `part1_surface_code/`). Now, we must find a workflow that ensures the preservation of a logical qubit under non-Clifford operations (rotation).

Beginning with the distance-3 patch and extending the geometry generation to alternative distances of$d$. Our implementation explicitly constructs the data-qubit layout. After understanding a clean baseline, we extended the circuit with different noise models including data noise known as the sitter, entangling noise, and measurement noise known as the mover. This allowed us to move beyond ideal stabilizer simulations and probe how distinct physical error sources influence detectors.

We simulate the STAR injection protocol using `tsim` across three code distances ($d = 3, 5, 7$) and a sweep of logical rotation angles θ spanning values from 10^{-4} to 0.5 radians, with physical depolarizing noise $p = 10^{-3}$ occurring with every operation in the STAR architecture. Using the outputs of the stabilizer circuits following the `tsim` simulations, we trained a linear classifier with cross-entropy gradient descent as the syndrome decode, predicting whether the logical qubit flipped or not. The full simulation workflow and fidelity reproduction process is documented in `fidelity_simulation.ipynb`

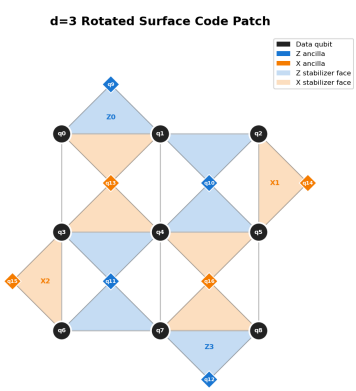


FIG. 6. Distance-3 rotated surface-code patch used in our analysis. Black nodes represent data qubits, while blue and orange ancillas correspond to Z and X stabilizers, respectively.

Finally, we modeled different simulations visualizing how expensive it is to run a 4-qubit QFT using STAR teleportation as code distance increases.

This validation and noise modeling stage ensures that subsequent resource estimates are grounded in a physically meaningful and correctly implemented encoded system.

B. Part 2: Estimating STAR Fidelities

The complete implementation and simulation of this logical gate teleportation protocol can be found in `part3.ipynb`.

1. *Physical Rotation Angle Mapping*

The STAR architecture applies d physical R_Z rotations transversally to the left column of data qubits. For a code of distance d (an odd integer), the required physical angle θ_{phys} satisfies:

$$\tan\left(\frac{\theta_{\text{phys}}}{2}\right)^d = \tan\left(\frac{\theta_{\log}}{2}\right)$$

which implies

$$\theta_{\text{phys}} = 2\arctan\left[\operatorname{sgn}(t)\,|t|^{1/d}\right], \quad t = \tan\left(\frac{\theta_{\log}}{2}\right),$$

with a sign $s = -1$ when $(d+1) \equiv 0 \pmod 4$, and $s = +1$ otherwise.

Listing 1. Physical angle computation provided by the Challenge

```
def physical_angle(logical_angle_in_pi: float, num_physical_rotations: int) -> float:
    assert num_physical_rotations % 2 == 1 and num_physical_rotations > 0
    sign = -1 if (num_physical_rotations + 1) % 4 == 0 else 1
    t = np.tan(logical_angle_in_pi * np.pi / 2)
    x = np.sign(t) * (np.abs(t) ** (1 / num_physical_rotations))
    return float(sign * 2 * np.arctan(x) / np.pi)
```

2. Circuit Structure

Each `.stim` circuit file encodes the following sequence:

1. **Initialise** data qubits in $|+\rangle$ via `RX`.
2. **Apply** `R_Z`(θ_{phys}) transversally to the left column (d qubits), producing the encoded ancilla $|m_\theta\rangle$.
3. **Two syndrome rounds** with post-selection on all syndromes $= 0$.
4. **Feed-forward** (replaced by post-selection in simulation) to correct byproduct operators from state preparation.
5. d **further syndrome rounds** to track the stabilised code.
6. **Perfect logical unrotation**: $R_Z(-\theta_{\log})$ applied via the logical chain in the noiseless final block.
7. **Logical X measurement**: `MPP X0*X1*X2` with `OBSERVABLE_INCLUDE(0)`.

The relevant tsim lines for $d = 3$ are:

```
R_Z({phys_angle}) 0 3 6        # transversal rotation, left column
CX 0 3  3 6                    # logical unrotation chain
R_Z({-logical_angle}) 6        # perfect unrotation (noiseless block)
CX 3 6  0 3
MPP X0*X1*X2                   # measure logical X
OBSERVABLE_INCLUDE(0) rec[-1]
```

3. Post-Selection

Since `tsim` does not support feed-forward at sampling time, post-selection is applied in Python after sampling: only shots where the first k detectors are all zero are retained.

```
k    = min(first_k_detectors, detectors.
    shape[1])
keep = ~np.any(detectors[:, :k], axis=1)
```

The post-selection depth k was set to 20, 120, and 300 for $d = 3$, 5, and 7 respectively. The purpose was to ensure that the act of rotating the physical qubits did not cause infidelity in the system. If early measurements in this phase of the simulations indicate infidelity, then it is unlikely that the rest of the system will remain logical.

4. Decoder: Linear Classifier with Cross-Entropy Loss

After post-selection, the detector array for each surviving shot is used to predict whether a logical error occurred. We use logistic regression trained with cross-entropy loss, which directly minimizes prediction error.

Listing 2. Cross-entropy gradient and training loop

```
def cross_entropy_grad(X, y, w, b):
    p   = expit(np.clip(X @ w + b, -500, 500))
    return X.T @ (p - y) / len(y), (p - y).mean()

for epoch in range(epochs):
    gw, gb = cross_entropy_grad(X_train, y_train, w, b)
    w -= lr * gw;  b -= lr * gb
    norm = np.linalg.norm(w)
    if norm > max_weight_norm:
        w *= max_weight_norm / norm   # prevent weight explosion
```

5. Results

Hyperparameter	Symbol	$d=3$	$d=5$	$d=7$
Shots per angle	n_{trials}	5000	5000	5000
Post-selection depth	k	20	120	300
Learning rate	η	0.01	0.01	0.01
Training epochs		2000	5000	5000

6. Logical Error Rate vs. Rotation Angle

This can be seen in Figure 7.

7. Discussion

a. Distance scaling. Across the full angle range, $\varepsilon_{d=7} < \varepsilon_{d=5} < \varepsilon_{d=3}$, confirming that larger codes improve logical fidelity (i.e. the model's ability to correct logical errors) as expected. Normalized by distance, $d = 7$ achieves error rates $\sim$3–4$\times$ lower than $d = 3$ in the small-angle regime.

b. Flat angle dependence at small θ. The error rate is approximately *constant* across $\theta \in [10^{-4}\pi,\ 10^{-1}\pi]$. The dominant error source is the depolarizing noise in syndrome extraction ($p = 10^{-3}$), not the rotation itself. The theoretical rotation-induced error $\sin^2(\theta\pi/2) \approx (\theta\pi/2)^2$ is negligibly small compared to the noise floor for $\theta \lesssim 0.1\pi$.

c. Large-angle degradation. At $\theta = 0.5\pi$, all distances show elevated error rates. The physical angle $\theta_{\text{phys}} = 2\arctan[(\tan(\pi/4))^{1/d}]$ is no longer small, and the post-selection survival rate drops as the injected state deviates further from a stabilizer eigenstate. The $d = 7$ curve is notably more robust here, with $\varepsilon \approx 6 \times 10^{-3}$, versus $\varepsilon \approx 1.9 \times 10^{-2}$ for $d = 3$.

C. Part 3: Gate Teleportation Protocol

1. Overview

The gate-teleportation approach to injecting $R_z(\theta)$ into the main logical qubit is illustrated in Figure 8.

Noiseless STAR gate-teleportation (no feed-forward version):

1. **Initialize** both patches in $|+\rangle_{\mathcal{L}}$ via transversal RX on all data qubits.
2. **Prepare resource state**: apply $R_z(\theta_{\text{phys}})$ to the left column of the *ancilla* patch ($d = 3$ physical qubits). This encodes $R_z(\theta_{\mathcal{L}})\,|+\rangle_{\mathcal{L}}$ in the ancilla logical space.
3. **Transversal CNOT**: data patch (control) $\rightarrow$ ancilla patch (target), one CX per corresponding pair of data qubits.
4. **Measure** all ancilla data qubits in the X basis (MX).
5. **Post-select** on ancilla logical $X = 0$ (i.e. the XOR of the first row of MX outcomes equals zero). This selects shots where $R_z(+\theta_{\mathcal{L}})$ was applied to the data patch.
6. **Read out** the data logical X via $\overline{X} = MPP\ X_0X_1X_2$.

2. Observable Assignment

After the MX of the 9 ancilla data qubits and the MPP on the data patch, the circuit defines two observables tracked by `tsim`:

Index	Definition	Interpretation
0	$MPP\ X_0X_1X_2$	Logical X of the data patch. After post-selection, $P(\text{obs}_0 = 1) = \sin^2(\theta_{\mathcal{L}}\pi/2)$.
1	$\bigoplus_{j\in\{17,18,19\}} m_j$	Logical X of the ancilla patch (XOR of first row). Post-select on $\text{obs}_1 = 0$.

D. Circuit Implementation

1. Qubit Layout

The two $d = 3$ STAR code patches use only their 9 data qubits each (no syndrome ancilla are needed in the noiseless setting):

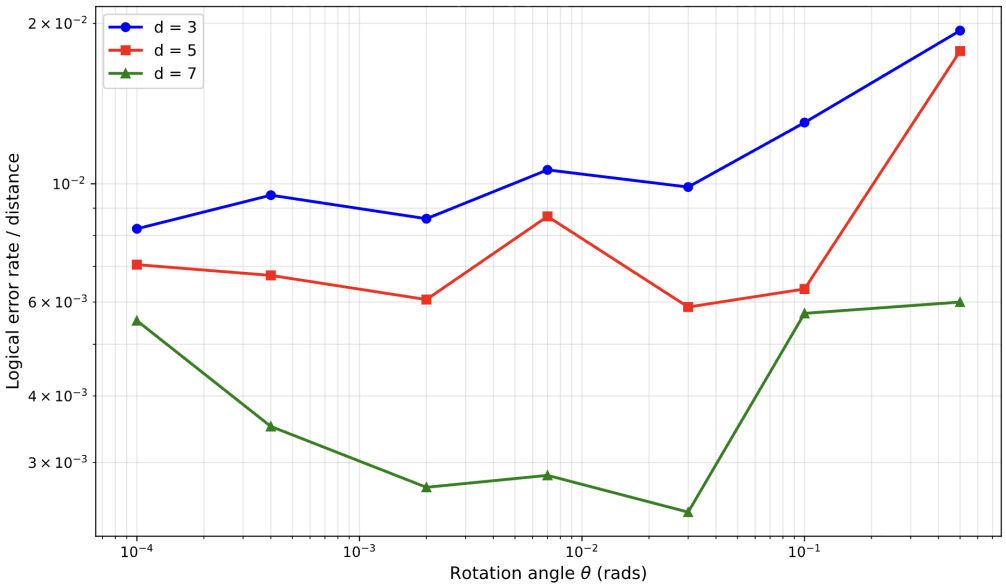


FIG. 7. Logical error rate divided by code distance as a function of rotation angle θ (in radians) for $d = 3$ (blue circles), $d = 5$ (red squares), and $d = 7$ (green triangles). Dividing by d normalises for the different numbers of syndrome checks per round.

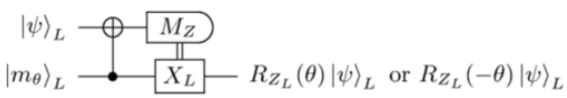
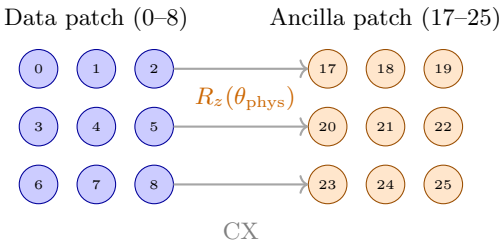


FIG. 8. The Repeat-Until-Success (RUS) circuit for STAR rotation injection. Figure from Akahoshi *et al.*, "Partially Fault-Tolerant Quantum Computing Architecture with Error-Corrected Clifford Gates and Space-Time Efficient Analog Rotations."



The left column of the ancilla patch (qubits 17, 20, 23) receives the physical rotation. The transversal CNOT connects every data-qubit pair $(i, i{+}17)$ for $i = 0, \ldots, 8$.

2. *Angle Sweep*

Sweeping $\theta_\mathcal{L}$ from 0.001π to 0.5π confirms that the post-selected data logical X error rate tracks $\sin^2(\theta_\mathcal{L}\pi/2)$ as predicted:

$\theta_\mathcal{L}/\pi$	Survival	Meas. error	Expected $\sin^2(\theta_\mathcal{L}\pi/2)$
0.001	$\approx 100\%$	≈ 0	2.5×10^{-6}
0.01	$\approx 94\%$	≈ 0	2.5×10^{-4}
0.1	$\approx 77\%$	≈ 0.0	0.024
0.2	$\approx 68\%$	≈ 0.095	0.095
0.3	$\approx 61\%$	≈ 0.206	0.206
0.5	$\approx 50\%$	≈ 0.500	0.500

The survival fraction decreases from $\approx 100\%$ (for $\theta_\mathcal{L} \to 0$, when the ancilla rotation is negligible and the measurement is almost always $+1$) toward 50% (for $\theta_\mathcal{L} = \pi/2$, when the ancilla is in $|i\rangle_\mathcal{L}$ and the X measurement is maximally random). This matches $\cos^2(\theta_\mathcal{L}\pi/2)$ for the first-row XOR.

3. *Cost Analysis*

RUS Overhead

Because `tsim` does not support feed-forward, we use **Repeat-Until-Success (RUS)** post-selection. Each attempt at injecting $R_z(\theta_\mathcal{L})$ succeeds with probability

$$p_{\text{success}} = \cos^2\left(\frac{\theta_\mathcal{L}\pi}{2}\right), \tag{7}$$

which equals the probability that the ancilla logical X measurement returns 0. The expected number of attempts before success is $1/p_{\text{success}}$.

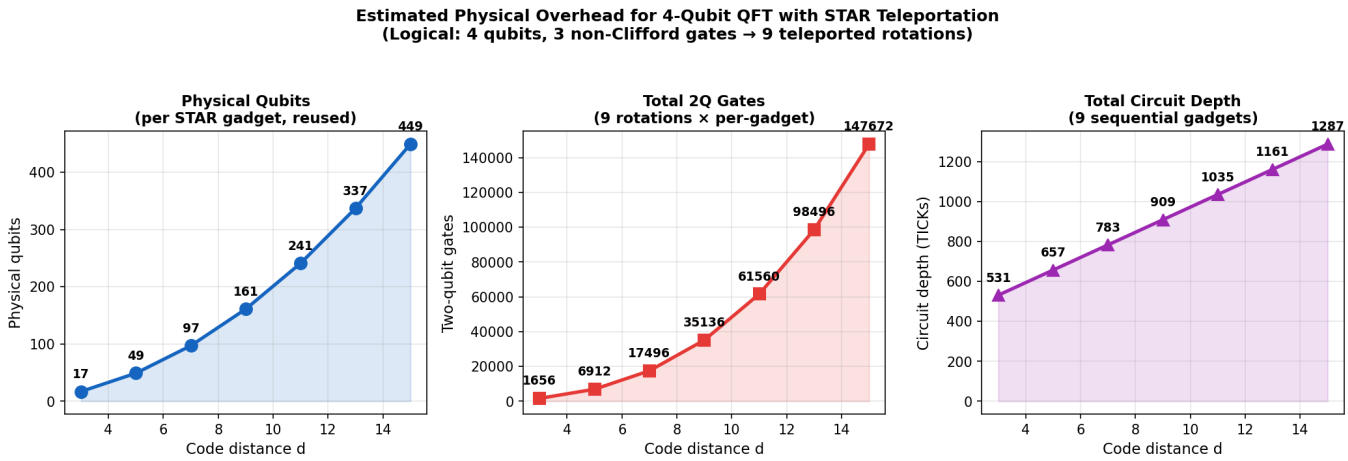


FIG. 9. Estimated physical overhead for a 4-qubit QFT implemented with STAR teleportation under surface-code protection. From left to right, the plots show the total number of physical qubits, total two-qubit gates, and total circuit depth as functions of code distance.

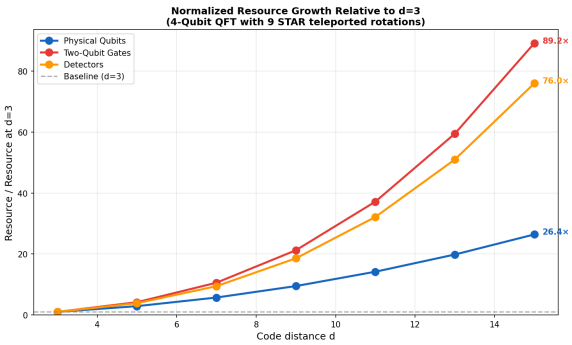


FIG. 10. Normalized resource growth relative to the $d = 3$ baseline for the 4-qubit QFT benchmark. Two-qubit gate overhead grows the fastest, followed by detector-related overhead, while physical qubit growth is slower but still significant.

$\theta_{\mathcal{L}}/\pi$	p_{success}	Avg. attempts
0.01	0.9998	1.0002
0.1	0.976	1.025
0.3	0.794	1.26
0.5	0.500	2.00
1.0	0.000	∞

Per-Rotation Resource Count

Each attempt uses the following resources:

Resource	Per attempt	Average (small θ)
Ancilla logical prep	1	$1/p_{\text{success}} \approx 1$
Physical rotations (R_Z)	$d = 3$	$3/p_{\text{success}} \approx 3$
Transversal CNOT gates	$d^2 = 9$	$9/p_{\text{success}} \approx 9$
X-basis measurements	$d^2 = 9$	$9/p_{\text{success}} \approx 9$
Logical X readout	1	1

In a real device there would be additional syndrome extraction rounds (approximately d rounds on the data patch after each injection, costing $O(d^3)$ two-qubit gates), but in the noiseless simulation these are omitted.

E. Part 4: Resource Scaling for Fault-Tolerant QFT

To study the system-level cost of fault-tolerant execution, we analyzed a 4-qubit Quantum Fourier Transform (QFT) protected by the surface code. This benchmark was chosen because it is small enough to study clearly, but still contains non-Clifford structure that must be implemented through STAR teleportation-based rotations.

In this setting, the logical QFT contains three non-Clifford gates, which expand into multiple teleported rotations at the encoded level. As code distance increases, each protected operation requires a larger patch, more stabilizer measurements, and greater gate overhead. We therefore tracked three key resource metrics as a function of code distance:

1. the total number of physical qubits,
2. the total number of two-qubit gates,
3. the total circuit depth.

Figure 9 shows that all three resource metrics increase substantially with code distance. Physical qubit count grows because larger code distances require larger encoded patches and associated ancilla structure. The total number of two-qubit gates grows even more rapidly due to repeated syndrome extraction and teleportation gadget execution. Circuit depth also increases steadily because these operations are applied sequentially in the encoded implementation.

To better isolate the scaling trend, we also normalized all resource counts relative to the $d = 3$ baseline.

The normalized view makes the main systems result especially clear: two-qubit gate overhead is the dominant scaling bottleneck. This is important because entangling gates are typically among the most expensive and noise-sensitive operations in a fault-tolerant architecture. Overall, these results show that increasing code distance improves protection, but introduces substantial overhead in gate count and execution time, emphasizing the need to optimize syndrome schedules, teleportation protocols, and encoded compilation strategies together.

VII. AI USAGE

The main tools we used were Claude Code as our coding agent and ChatGPT to help us understand different resources. Claude was majorly used for making plots, automating simulations,

[1] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, Cambridge University Press, 10th Anniversary Edition, 2010. Chapters 4.5 (universal gates) and 4.5.3 (two-qubit gates are necessary).

[2] A. Y. Kitaev, "Quantum computations: algorithms and error correction," *Russian Mathematical Surveys*, 52(6):1191–1249, 1997. (Solovay–Kitaev theorem: any dense subgroup of $SU(2)$ can approximate any element to precision ε with $O(\log^c(1/\varepsilon))$ gates.)

[3] C. M. Dawson and M. A. Nielsen, "The Solovay-Kitaev algorithm," *Quantum Information & Computation*, 6(1):81–95, 2006. arXiv:quant-ph/0505030.

[4] J.-L. Brylinski and R. Brylinski, "Universal quantum gates," in *Mathematics of Quantum Computation*, Chapman & Hall/CRC, 2002. arXiv:quant-ph/0108062. (Proves that any 2-qubit entangling gate plus all 1-qubit gates is universal, and that 1-qubit gates alone are not.)

[5] V. Kliuchnikov, D. Maslov, M. Michele, "Fast and efficient exact synthesis of single qubit unitaries generated by Clifford and T gates," arXiv:1206.5236